Name: Hasini

hasiniakkireddy@gmail.com

Date: 25.05.2026

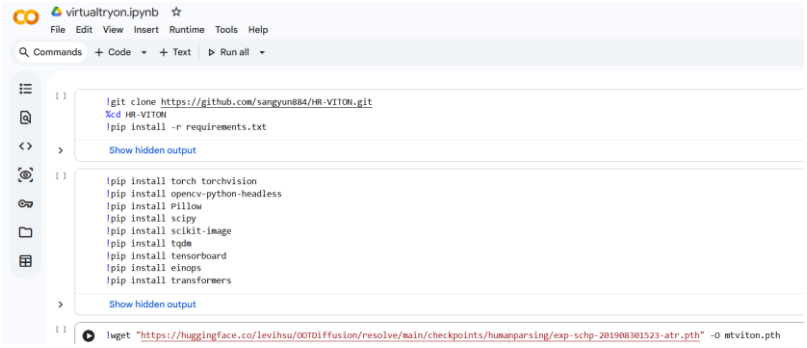
Role: AI/ML Intern

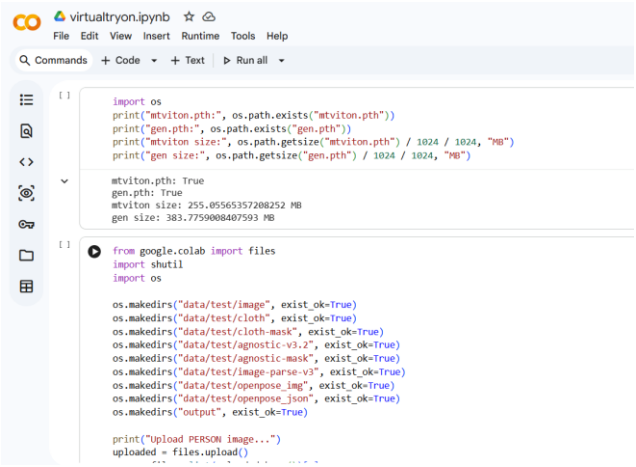
Task 1 — 2D Virtual Try-On

I worked on setting up the HR-VITON virtual try-on model on Google Colab to generate a realistic clothing try-on output.

Overview

I started by cloning the HR-VITON repository from GitHub on Google Colab and setting up all the required dependencies. HR-VITON requires two model weight files — mtviton.pth and gen.pth — which I downloaded separately. I verified both files were correctly downloaded by checking their sizes using Python's os module.



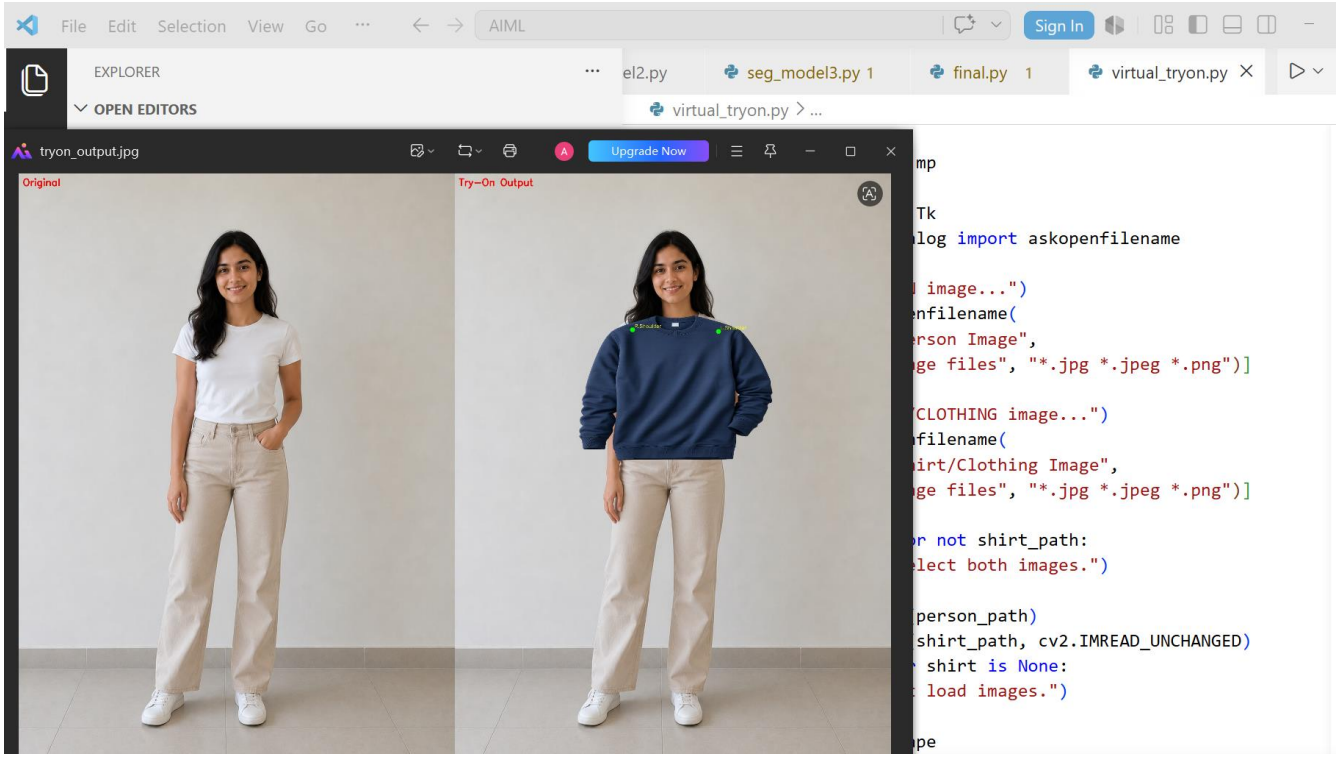


Challenges Faced

- MediaPipe had protobuf version conflicts with TensorFlow on Colab so real pose detection could not be run
- Since real OpenPose and DensePose preprocessing tools were not available the keypoints and body maps were estimated manually
- Because of manually approximated inputs the model output was distorted and not usable

- HR-VITON strictly requires accurate OpenPose keypoints and DensePose maps generated from the actual person image
- Manual estimation of these inputs caused the model to incorrectly understand body shape and clothing placement
- The output showed distorted clothing with blending artifacts.

Since the Google Colab output was not good due to incorrect preprocessing inputs, I ran a custom virtual try-on code locally in VS Code which gave a much better and clean output.



What I Observed

- Person image loaded correctly
- Body landmarks were detected accurately using MediaPipe
- Clothing was placed correctly on the torso with proper shoulder alignment
- The navy blue sweatshirt was overlaid cleanly on the person
- Output showed Original on left and Try-On Output on right
- The result looked natural with correct body proportions maintained

Code:

Module3 > virtual_tryon.py > ...

```python
import cv2
import mediapipe as mp
import numpy as np
from tkinter import Tk
from tkinter.filedialog import askopenfilename
Tk().withdraw()
print("Select PERSON image...")
person_path = askopenfilename(
    title="Select Person Image",
    filetypes=[("Image files", "*.jpg *.jpeg *.png")]
)
print("Select SHIRT/CLOTHING image...")
shirt_path = askopenfilename(
    title="Select Shirt/Clothing Image",
    filetypes=[("Image files", "*.jpg *.jpeg *.png")]
)
if not person_path or not shirt_path:
    print("Please select both images.")
    exit()
person = cv2.imread(person_path)
shirt  = cv2.imread(shirt_path, cv2.IMREAD_UNCHANGED)
if person is None or shirt is None:
    print("Could not load images.")
    exit()
h, w, _ = person.shape
mp_pose    = mp.solutions.pose
pose       = mp_pose.Pose(
    static_image_mode=True,
    model_complexity=2,
    min_detection_confidence=0.5
)
rgb        = cv2.cvtColor(person, cv2.COLOR_BGR2RGB)
results    = pose.process(rgb)
pose.close()
if not results.pose_landmarks:
    print("No pose detected. Use a clear full body image.")
    exit()
```

Module3 > virtual_tryon.py > ...

```python
lm         = results.pose_landmarks.landmark
l_shoulder = lm[mp_pose.PoseLandmark.LEFT_SHOULDER]
r_shoulder = lm[mp_pose.PoseLandmark.RIGHT_SHOULDER]
l_hip      = lm[mp_pose.PoseLandmark.LEFT_HIP]
r_hip      = lm[mp_pose.PoseLandmark.RIGHT_HIP]
ls_x = int(l_shoulder.x * w)
ls_y = int(l_shoulder.y * h)
rs_x = int(r_shoulder.x * w)
rs_y = int(r_shoulder.y * h)
lh_y = int(l_hip.y * h)
shirt_width  = int(abs(ls_x - rs_x) * 2.8)
shirt_height = int(abs(lh_y - ls_y) * 1.8)
shirt_x = int((rs_x + ls_x) / 2) - shirt_width // 2
shirt_y = int((ls_y + rs_y) / 2) - int(shirt_height * 0.25)
shirt_x = max(0, shirt_x)
shirt_y = max(0, shirt_y)
if shirt_x + shirt_width > w:
    shirt_width = w - shirt_x
if shirt_y + shirt_height > h:
    shirt_height = h - shirt_y
shirt_resized = cv2.resize(shirt, (shirt_width, shirt_height))
output = person.copy()
if shirt_resized.shape[2] == 4:
    shirt_rgb   = shirt_resized[:, :, :3]
    shirt_alpha = shirt_resized[:, :, 3] / 255.0
    for c in range(3):
        output[shirt_y:shirt_y + shirt_height,
               shirt_x:shirt_x + shirt_width, c] = (
            shirt_alpha * shirt_rgb[:, :, c] +
            (1 - shirt_alpha) * person[shirt_y:shirt_y + shirt_height,
                                       shirt_x:shirt_x + shirt_width, c]
        )
else:
    shirt_gray  = cv2.cvtColor(shirt_resized, cv2.COLOR_BGR2GRAY)
    _, shirt_mask = cv2.threshold(shirt_gray, 240, 255, cv2.THRESH_BINARY_INV)
    shirt_mask  = shirt_mask / 255.0
    for c in range(3):
        output[shirt_y:shirt_y + shirt_height,
```

Module3 > virtual_tryon.py > ...

```python
            output[shirt_y:shirt_y + shirt_height,
                   shirt_x:shirt_x + shirt_width, c] = (
                shirt_mask * shirt_resized[:, :, c] +
                (1 - shirt_mask) * person[shirt_y:shirt_y + shirt_height,
                                          shirt_x:shirt_x + shirt_width, c]
            )
cv2.circle(output, (ls_x, ls_y), 6, (0, 255, 0), -1)
cv2.circle(output, (rs_x, rs_y), 6, (0, 255, 0), -1)
cv2.putText(output, "L.Shoulder", (ls_x + 5, ls_y - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 255), 1)
cv2.putText(output, "R.Shoulder", (rs_x + 5, rs_y - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 255), 1)
combined = cv2.hconcat([person, output])
cv2.putText(combined, "Original", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
cv2.putText(combined, "Try-On Output", (w + 10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
cv2.imwrite("tryon_output.jpg", combined)
cv2.imshow("Virtual Try-On Demo", combined)
cv2.waitKey(0)
cv2.destroyAllWindows()
print("Done! Output saved as tryon_output.jpg")
```

Task 2 — 3D Virtual Try-On Research

Today I researched how 3D virtual try-on systems work and explored the tools and technologies used to build them.

- 3D virtual try-on goes beyond simple image overlay
- It creates a full 3D body model of the person and simulates how clothing fits on it
- The output looks realistic from all angles and responds to body movement
- It is more complex than 2D try-on but gives much more realistic results

3D Avatars

- A 3D avatar is a digital human body model that represents the person in three dimensions
- Avatars can be generated from a single photo or body measurements
- They capture body shape, size and proportions accurately

Body Mesh Generation

- A body mesh is a 3D structure made of triangles that covers the human body surface
- Each triangle is called a polygon and together they form the complete body shape
- SMPL model is the most popular body mesh model used in research

Cloth Simulation

- Cloth simulation is the process of making virtual fabric behave like real fabric
- It handles how fabric stretches, folds, wrinkles and drapes on the body

Real-time Fitting Systems

- Real-time fitting means the clothing updates instantly as the person moves
- This is used in AR try-on applications where the camera shows the person live
- Real-time systems need to be very fast and run at 30 frames per second or more

Motion Tracking

- Motion tracking detects how the body is moving in real time
- It tracks all body joints like shoulders, elbows, hips and knees
- MediaPipe and OpenPose are used for 2D motion tracking
- For 3D motion tracking depth cameras like Intel RealSense are used
- Motion data is fed into the avatar so it moves exactly like the real person
- The clothing simulation then updates based on the avatar movement

Tools Explored

- **Blender** — open source 3D software with powerful cloth simulation, used for creating and testing 3D garments
- **Unity** — game engine used for real-time 3D try-on applications and AR experiences
- **Three.js** — JavaScript library for rendering 3D graphics in the browser, useful for web based try-on

- **SMPL Model** — statistical human body model that generates realistic body meshes from shape and pose parameters
- **ARKit and ARCore** — AR frameworks from Apple and Google for real-time mobile try-on experiences

Research Findings

- 3D try-on is significantly more complex than 2D try-on
- It requires body mesh generation, cloth simulation and real-time rendering all working together
- SMPL model is the best starting point for body representation
- Blender is best for offline high quality cloth simulation
- Unity and Three.js are best for real-time and web based applications
- Full 3D try-on systems require high GPU power for real-time use

Task 3 — AI Model Exploration

Human Pose Estimation

- Process of detecting body joint positions from an image
- Outputs keypoints for shoulders, elbows, hips, knees and ankles
- Used to understand body position so clothing can be aligned correctly

Body Landmark Detection

- Detects precise locations of up to 33 body points
- Used to calculate body measurements like shoulder width and torso height
- Helps in selecting correct clothing size and fit

AI Fashion Recommendation Integration

- Suggests clothing based on body type, occasion and style preference
- Can be rule based or machine learning based
- Combining recommendation with try-on creates a complete fashion experience

Segmentation Models

- Identifies which pixels belong to which body part or clothing
- Used to separate clothing from skin, hair and background
- Tells the model exactly where to place new clothing

MediaPipe Pose

- Developed by Google
- Detects 33 body landmarks in real time
- Very fast, works on mobile without GPU
- Used in my body measurement and try-on prototype

Detectron2

- Developed by Facebook AI Research
- Supports pose estimation and body part segmentation
- More powerful but needs more computing resources
- Used for detailed body parsing in try-on pipelines

U2Net

- Specialized for background removal and clothing mask generation
- Very accurate at separating clothing from background
- Better than simple thresholding for generating cloth masks

OpenPose

- Detects 25 body keypoints in standard format
- More accurate than MediaPipe for complex poses
- I generated OpenPose format JSON manually for HR-VITON pipeline today

Task 4 — Dataset Collection

I researched datasets used in virtual try-on and fashion AI systems.

Fashion Clothing Images

- **DeepFashion** —800k clothing images, used for clothing recognition and recommendation
- **iMaterialist** — Kaggle — clothing attribute dataset, used for clothing type detection
- **Fashionpedia** — fashionpedia.github.io — 48k fashion images with segmentation masks, used for clothing detection

Human Pose Datasets

- **COCO Keypoints** — cocodataset.org — 200k images with 17 body keypoints, used for training pose estimation models
- **Human3.6M** — vision.imar.ro — 3.6 million frames with 3D poses, used for 3D body reconstruction

Body Segmentation Datasets

- **ATR Dataset** — HuggingFace — 17,000 images with 18 body part labels, used for body parsing

Apparel Try-On Datasets

- **VITON Dataset** — github.com/xthan/VITON — 16,000 person and clothing image pairs, used for training 2D try-on models
- **VITON-HD Dataset** — github.com/shadow2496/VITON-HD — 13,000 high resolution image pairs, used for training HR-VITON